CookMem Tutorial 2

Complex Pointer Graphs

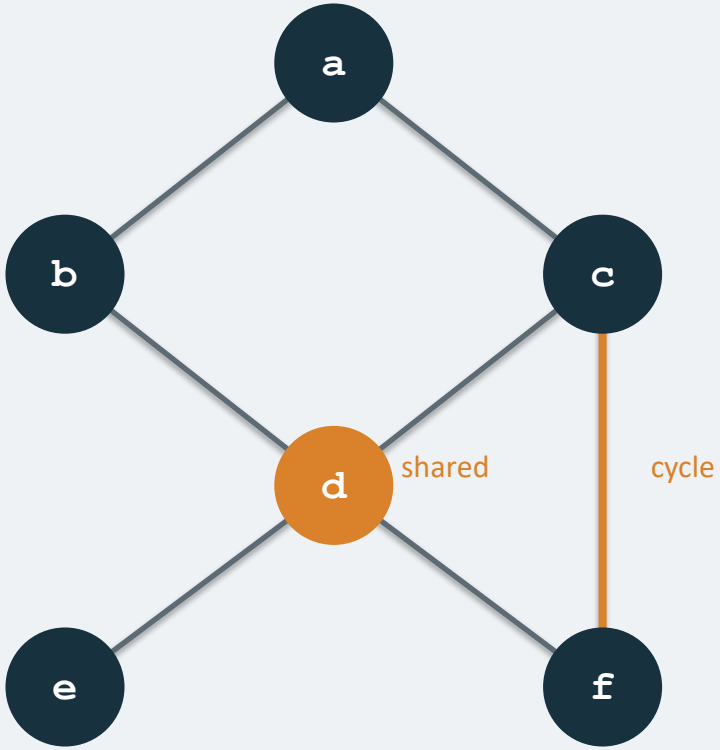
Build big object graphs, and drop them in one call



For compilers, parsers, database engines and request-scoped servers

Who frees what?

Shared children and a cycle



`std::shared_ptr`

Work on every pointer copy, and cycles keep themselves alive: a leak.

`delete by hand`

Every node exactly once, after nothing points to it, on every error path too.

A garbage collector

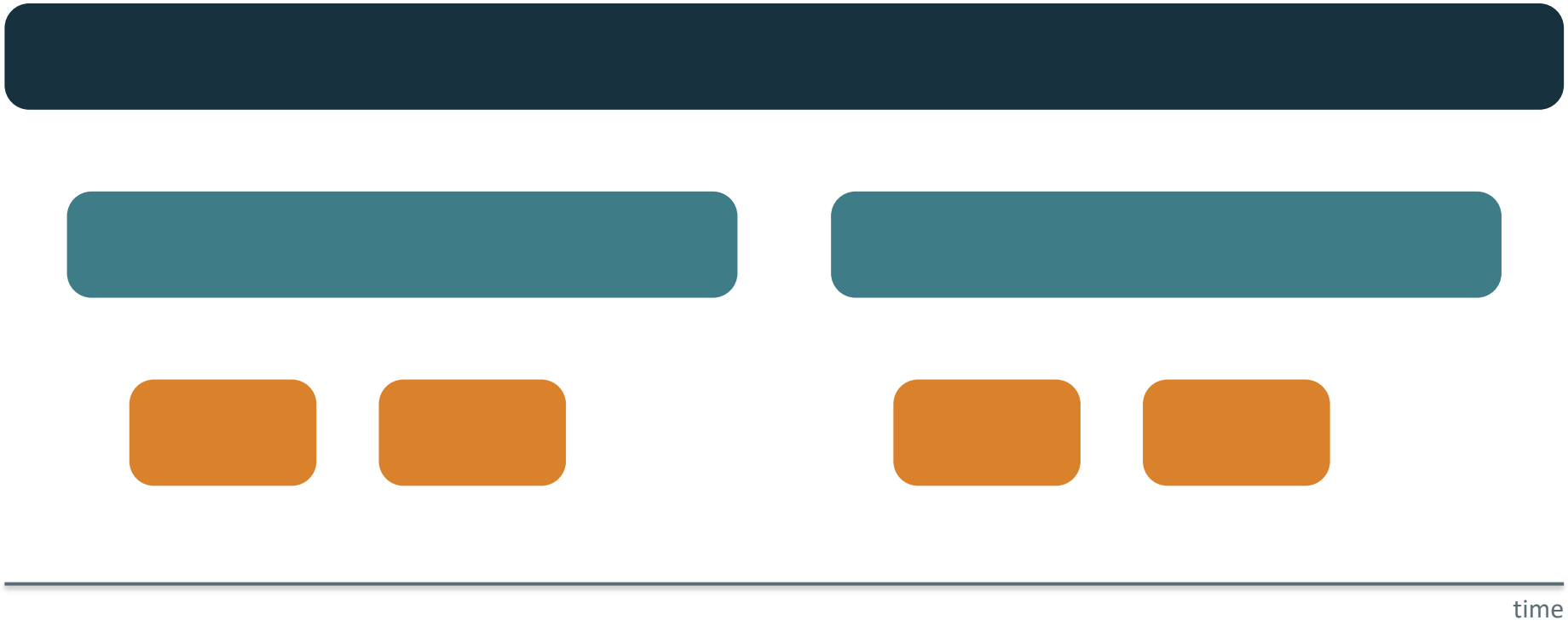
Solves it, but C++ does not have one built in.

One context per lifetime

Session
symbol tables, settings

Compilation unit
syntax tree, types

Optimizer pass
temporary graphs



Each bar is a context. Allocate every object from the context of its lifetime, and release the context when that lifetime ends.

Creating objects in a context

```
// Allocate and construct, like new.
Node* node = memCtx.create<Node> (memCtx, Node::NUMBER);

// Run the destructor and free, like delete.
memCtx.destroy (node);

// Or free every object at once, without destructors.
memCtx.releaseAll ();
```

CookMem 1.6

1. **create<T> ()**
   Aligned for T; throws bad_alloc, like new.

2. **Context first**
   Objects can put their strings and lists there too.

3. **destroy ()**
   Frees one object; releasing frees them all.

Example: an expression tree

```
Context memCtx;
const char* text = "max(1, 2 * (3 - x)) + -y / 4";
Node* tree = Parser (memCtx, text).parse ();

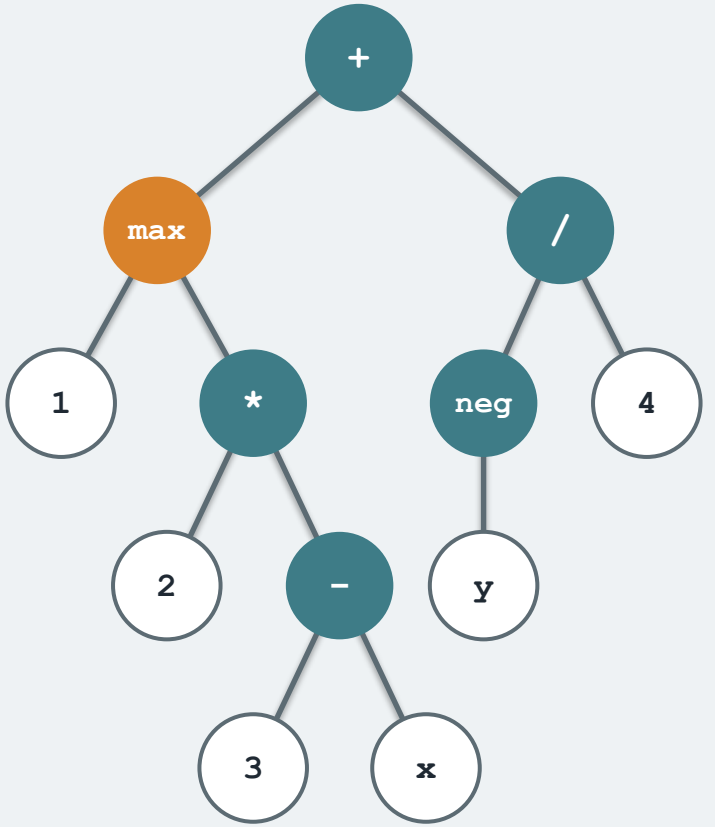
print (std::cout, tree);
std::cout << evaluate (tree, vars) << "\n";    // 2

memCtx.releaseAll ();   // no tree walk, no delete
```

examples/01_expression_ast.cpp (shortened)

18 allocations, one release. Nodes, names and child lists all come from memCtx, so releaseAll () frees every one of them.

```
try { Parser (memCtx, "1 + (2 * ").parse (); }
catch (const std::runtime_error& ex) { /* report */ }
memCtx.releaseAll ();   // half-built nodes go too
```



Containers inside nodes

```
struct Node
{
    enum Kind { NUMBER, VARIABLE, NEGATE, BINARY, CALL };

    Kind        kind;
    double      value;      // NUMBER
    char        op;         // BINARY: + - * /
    String      name;       // VARIABLE and CALL
    List<Node*> children;   // operands or arguments

    Node (Context& memCtx, Kind k)
    : kind (k), value (0), op (0),
      name (cookmem::Allocator<char> (memCtx)),
      children (cookmem::Allocator<Node*> (memCtx))
    {
    }
};
```

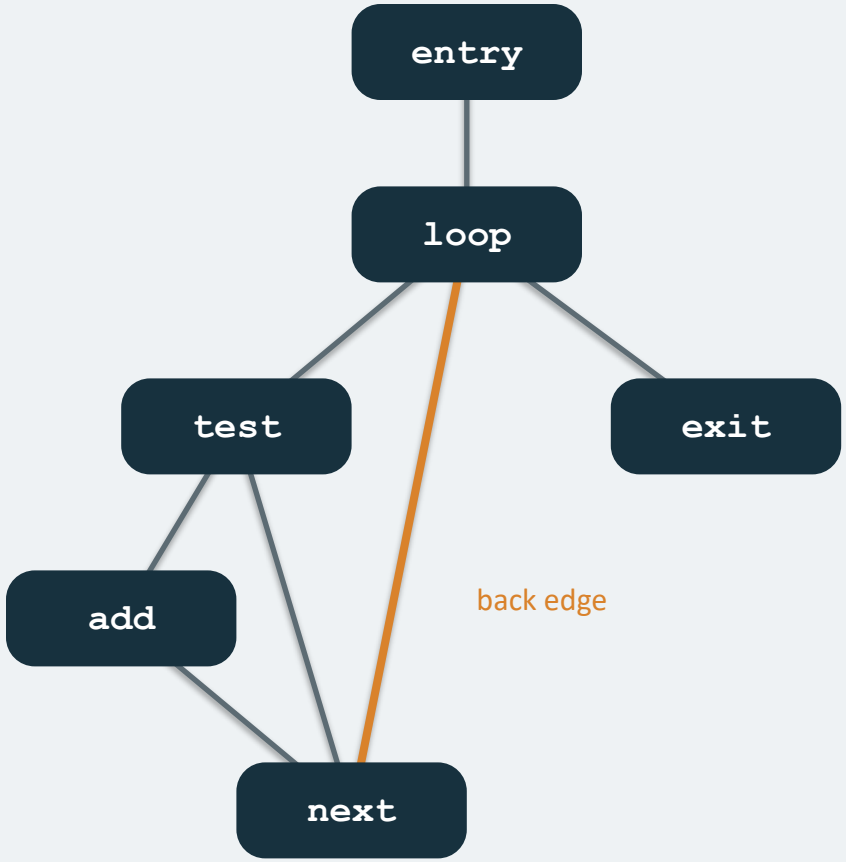
examples/ast.h (shortened)

1. **String and List**
   std::basic_string and std::vector with cookmem::Allocator.

2. **Same context**
   The tree owns only context memory.

3. **Errors are free**
   Half-built trees go with the same release.

Cycles are fine




```
Block* loop = memCtx.create<Block> (memCtx, "loop");
Block* next = memCtx.create<Block> (memCtx, "next");
// ...
link (next, loop);   // the back edge

memCtx.releaseAll ();   // frees the graph
```

examples/02_cyclic_graph.cpp (shortened)

std::shared_ptr	the cycle leaks
delete	each block exactly once
context	**one call**

Keeping results

```
Context session;            // lives for the whole program
{
    Context work (session, true);   // shares cached segments

    Node* tree = Parser (work, text).parse ();
    double value = evaluate (tree, vars);

    // Copy the result out before work goes away.
    definitions.push_back (
        session.create<Definition> (session, name, value));

    session.contains (definitions[0]);  // true
    work.contains (definitions[0]);     // false
}   // work is destroyed: every tree is freed at once
```

examples/03_keep_results.cpp (shortened)

1. **Short-lived work**
   Trees and temporaries go in work.

2. **Copy out**
   Results are made in the session.

3. **contains ()**
   Tells which context a pointer is in.

Graphs that change

```
// Replace (a + b) with one number node...
Node* result = number (memCtx, evaluate (node, vars));

// ...and free the three nodes it replaces.
memCtx.destroy (node->children[0]);
memCtx.destroy (node->children[1]);
memCtx.destroy (node);      // also frees its child list
```

examples/04_rewrite_in_place.cpp (shortened)

Freed nodes are reused. A bump allocator, which can only free everything at once, would grow with every edit.

Footprint after

1 edit

64 KB

1,000 edits

64 KB

100,000 edits

64 KB

One context per request

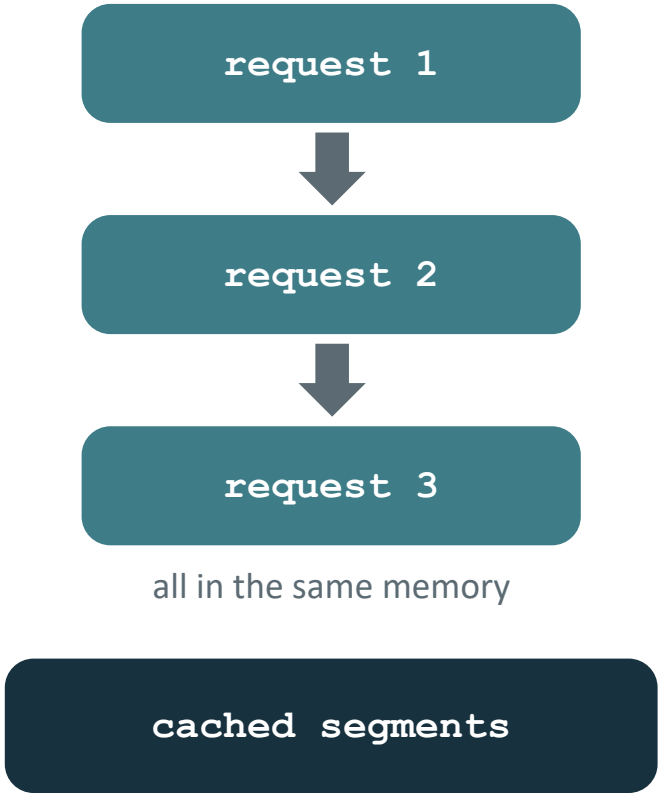
```
Context memCtx;     // a CachedMemContext

for (const char* request : requests)
{
    double result = handleRequest (memCtx, request, vars);
    // ... send the result ...

    memCtx.releaseAll ();   // segments stay in the cache
}
```

examples/05_per_request.cpp (shortened)

No system calls after the first request. 1,000 rounds of requests in the example all run in the same memory.



Pitfalls to avoid

Global-heap members

A plain std::string leaks on release. Use String and List from the context.

Pointers that outlive the context

Copy results out before you release.

One graph, two contexts

Releasing one leaves dangling pointers in the other.

Files, locks and sockets

No destructors run: destroy those objects first.

Summary

1. **One context per lifetime**
   Allocate a graph from one context and release it in one call, cycles and all.

2. **Keep only the results**
   Copy what must survive into a longer-lived context first.

3. **Free single nodes when the graph lives on**
   destroy () returns memory to the context for reuse.



Next: Tutorial 3, Single-Threaded Performance · github.com/coconut2015/cookmem-tutorials